

MagicQ miniQ Wing

This handbook describes building your own lighting console modeled after the modern **MagicQ Compact Mini Connect layout**. The console controls the MagicQ software via **keyboard commands (HID emulation)** using the software's built-in playback shortcuts function.

Note: The firmware uses a **3×13 button grid**. This handbook documents the keymap according to the **verified** state (checked via the firmware's diagnostic dump). The labeling of the individual keys can follow your own needs – what matters is the cell → key code mapping in the table below.

Table of contents

- 1. The layout (physical arrangement)
- 1.1 Firmware keymap (verified)
- 1.2 Fader calibration (no re-flashing needed)
- 2. Required hardware & shopping list
- 3. Hardware wiring
- 3.1 The faders (via the 74HC4067 multiplexer)
- 3.2 The key matrix (3x13 matrix) & diode wiring plan
- 4. Installation & flashing the bootloader
- Variant A: CircuitPython (`firmware/code.py`)
- Variant B: PlatformIO / C++ (`platformio.ini` in the project root)
- 5. Setup in MagicQ
- Project structure

1. The layout (physical arrangement)

The console has **3 rows of 13 buttons** and the fader section below them. At the top: the **S row** (select playback) with **MASTER GO** far right. Middle: the **GO row**. Bottom: the **flash row (F1–F10)** with **SWOP/PREV** on the left and **MASTER PAUSE** far right.

[MINI CONNECT WING]												
ALT	NEXT	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	MASTER GO
DBO	NEXT	G01	G02	G03	G04	G05	G06	G07	G08	G09	G010	RELEASE
SWOP	PREV	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	MASTER PAUSE
Grand			[Fader]		[Fader]	[Fader]	[Fader]	[Fader]	[Fader]	[Fader]	[Fader]	[Fa
Master			1		2	3	4	5	6	7	8	9

1.1 Firmware keymap (verified)

The mapping is defined in `src/main.cpp` as `KEYMAP[3][13]` and checked via the diagnostic combination `S10+GO10` (which types the cell table into the editor):

Cell	Key(s)	Key code	Function in MagicQ "Playback shortcuts"
0,0	ALT	<code>Ctrl+Alt+0</code> (hold)	Dead Black Out (DBO) short-circuit
0,1	NEXT	<code>[</code>	Next playback page
0,2–0,11	S1–S10	<code>1 2 3 4 5 6 7 8 9 0</code>	Select Playback 1–10
0,12	MASTER GO	<code>Space</code>	Manual GO
1,0	DBO	<code>F11</code>	Dead Black Out (Mac keyboard layout)
1,1	NEXT	<code>[</code>	Next playback page (2nd key)
1,2–1,11	GO1–GO10	<code>q w e r t y u i o p</code>	GO Playback 1–10
1,12	RELEASE	<code>-</code>	Release Playback
2,0	SWOP	<code>`</code> 	Add/Swap
2,1	PREV	<code>]</code>	Previous playback page
2,2–2,11	F1–F10	<code>\ z x c v b n m , .</code>	Flash Playback 1–10 (momentary)
2,12	MASTER PAUSE	<code>#</code>	Manual STOP

Special firmware behaviors:

- **Flash (F1–F10):** MagicQ "Test" keys toggle the playback to 100% – the firmware therefore sends a key press (**on**) when pressed and again (**off**) when released. Result: a **momentary** flash while your finger is on the key.
- **S + GO (same playback) = PAUSE/STOP:** While `S_n` and `GO_n` are held together, the firmware sends the **STOP key** (`a s d f g h j k l ;`). STOP is a toggle in MagicQ – every press pauses/starts. To avoid a "ws" ghost key press, each S/GO key is delayed ~40 ms; the chord detection decides whether STOP or the individual key is sent.
- **ALT:** held like a modifier (`Ctrl+Alt+0`) until the key is released.
- **MASTER GO / MASTER PAUSE:** in this layout they sit top right (GO) or bottom right (PAUSE) instead of being stacked vertically.

Fader handling (direct drag on the on-screen faders):

- The wing runs entirely in MagicQ **Playback shortcuts** keyboard mode (see section 5). The faders no longer send binary Test-key triggers. Instead they **grab and drag** the on-screen faders of the MagicQ **Full Panel** with the same absolute pointer mechanism as the sibling encoder wing:
- A second HID device is registered — an **absolute mouse** (digitizer-style, `ordering 8`, so it precedes the keyboard reports; the pointer maps the whole screen to `X=0..32767 / Y=0..32767`).
- Per fader the firmware jumps the cursor onto the fader track (button down), follows the physical fader, and releases again. Index 0 = **Grand Master**, 1..10 = **PB1..PB10**.
- Output modes via `FADER_OUT_MODE`: **2 = DRAG** (current default), **1 = SET** (keyboard entry of the value), **0 = WHEEL** (calibrated wheel nudges).

- **Ghosting filter (Richtungs-Persistenz):** a read value is only accepted proportionally after the next scan either continues in the same direction or holds near the candidate target; genuine A→B→A flicker is discarded, so nothing is sent while real faders are idle. Large single-scan jumps (`FADER_MAX_DELTA_PCT` , 90) and a deadband (`FADER_DEADBAND` , 0) protect against artifacts.
- **Idle/Locked faders:** unwired multiplexer inputs float (settle-dependent readings: 30 μs → ~1 %, 3000 μs → mid, plus a ~2 s charge-up and drift). `FADER_IDLE_GUARD` marks a channel abnormal at boot when short-settle ≠ long-settle reading (`FADER_IDLE_SPREAD`), and `FADER_FORCE_LOCK` deterministically locks the unwired channels (currently f03–f07) — locked channels keep their reference but never emit anything.
- Faders are read with `FADER_SETTLE_US` = 1500 μs and averaged (`FADER_SAMPLES` = 12, every 5th scan skipped); both ends snap to exactly 0/100 within `FADER_ENDSTOP` (1 %).
- On boot a **fader ADC scan** prints every mux channel (`ch<no> raw=<adc> <pct>%`) to verify the 74HC4067 wiring.
- All knobs (mode, drag gain, sampling, locks) are tunable at the top of `src/main.cpp` .

1.2 Fader calibration (no re-flashing needed)

Restart the Pico while holding the DBO button → calibration mode starts automatically (4 LED blinks). The cursor is steered with the wing buttons, and **NEXT Page saves** the values permanently in the Pico's LittleFS flash (`/fcal.bin`).

Key	Function
S1	X position of PB1
S2	X position of PB10
S3	Y of the PB track at 100 % (top) — cursor on the knob
S4	Y of the PB track at 0 % (bottom) — cursor on the knob
S5	X position of the Grand Master
S6	Y of the GM track at 100 % — cursor on the knob
S7	Y of the GM track at 0 % — cursor on the knob
F1 / F2	−500 / −50 (coarse / fine)
F3 / F4	+50 / +500
NEXT Page	save to flash and return to normal operation

PB2–PB9 are interpolated automatically between X(PB1) and X(PB10). Values are read back on every boot (`[CAL]` flash ... , or `[CAL] default ...` if no file exists yet). The compiled-in defaults of the current MagicQ Full Panel are: GM X = 400 (left of the Sub Master), PB X = 2800...25300 in steps of 2500, PB Y = 26800/29300, GM Y = 26850/29400.

Fader endstops (0 % / 100 %): A 10 k slide pot only covers part of the ADC range (tolerance, end caps, wiper end position), so without correction the top end stays below 100 % and the on-screen fader never reaches 100 %. Press **S8** to switch to the endstop mode:

Key	Function
S8	toggle between geometry mode and endstop mode
DBO / GO1... GO10	select the fader (DBO = Grand Master, GO1...GO10 = PB1...PB10)
S9 / S10	edit the lower (LO) or upper (HI) raw value of the selected fader
MASTER GO	take over the fader's currently measured raw value — push the fader fully down, press it for LO, fully up, press it again for HI
F1...F4	adjust the raw value manually (± 500 / ± 50)

[RAW] ... ist=... → ...% shows the live raw reading and the resulting percentage; a wrong endstop is immediately visible. Raw values are stored alongside the coordinates in `/fcal.bin`.

Wheel fallback (`FADER_OUT_MODE = 0`): one wheel report = 0.40 % travel, globally paced to one report per 25 ms because MagicQ evaluates at most ~40 reports/s. Kept as a backup for setups without a Full Panel.

Diagnostics combos:

- **S10 + GO10** (keep both held): types the currently pressed matrix cells as `M R2C3 R0C1` into the focused editor.
- **S1 + GO1**: types the current fader levels as `F GM=42% PB1=87% ...`.

2. Required hardware & shopping list

- **1x Raspberry Pi Pico** (standard or Pico H).
- **1x 74HC4067 multiplexer board** (16-channel analog multiplexer for reading the faders).
- **11x slide potentiometers (faders):**
- **Recommendation:** Bourns PTA6043 or ALPS RS6011Y.
- **Specification: 10k Ohm, LINEAR** (Important: labeled "B10K" or "10K Lin"). **60mm** travel corresponds to the original.
- **38x mechanical key switches** (3×13 grid, 38 cells populated, 1 spare):
- **Recommendation:** Cherry MX Black or Cherry MX Red (linear feel without audible clicking for fast, silent flashing).
- **38x keycaps:** standard 1U size. Suggested color scheme: gray (S), blue (Flash), green (GO).
- **38x diodes:** model **1N4148** (indispensable for avoiding "ghosting" in the key matrix).

3. Hardware wiring

3.1 The faders (via the 74HC4067 multiplexer)

The Pico has only 3 analog inputs (ADC), but we need 11 inputs for 10 playbacks + 1 grand master. The multiplexer switches these extremely quickly onto a single pin.

- **Fader power:** connect the left pin of all 11 faders to **3V3(OUT)** (Pico pin 36) and the right pin of all faders to **GND** (e.g. Pico pin 38).

- **Fader signal lines:** the middle pin (wiper) of faders 1 to 10 goes to inputs **C1 to C10** of the multiplexer. The grand master fader goes to the first input **C0**.
- **Multiplexer to Raspberry Pi Pico:**
- **VCC** -> 3V3 (Pico pin 36)
- **GND** -> GND (Pico pin 38)
- **SIG / COM** -> **GP27** (Pico pin 32 / ADC1)
- **S0** -> **GP17** (Pico pin 22)
- **S1** -> **GP18** (Pico pin 24)
- **S2** -> **GP19** (Pico pin 25)
- **S3** -> **GP20** (Pico pin 26)

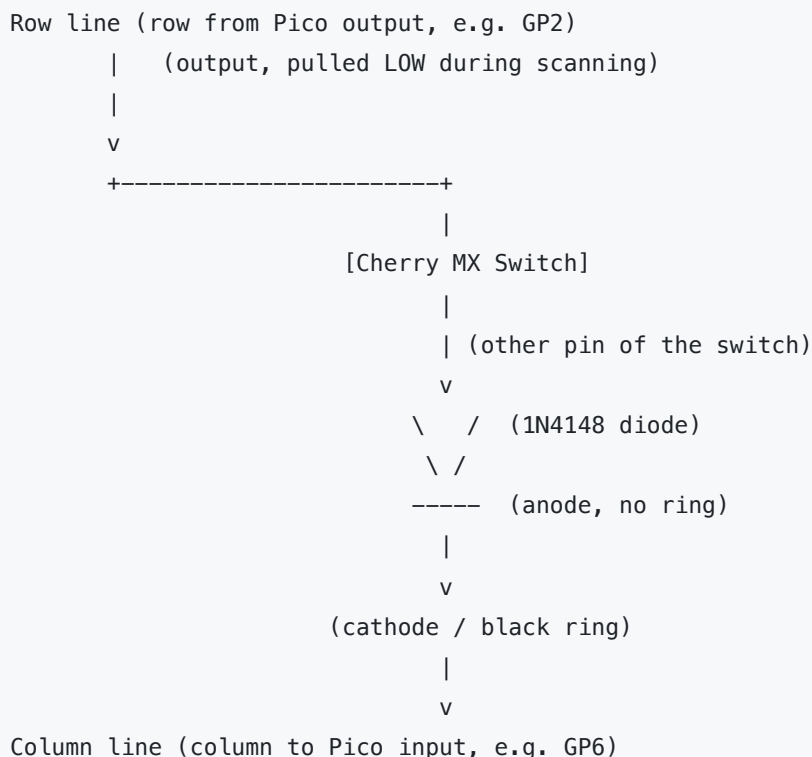
3.2 The key matrix (3x13 matrix) & diode wiring plan

To save pins, we wire the keys in 3 rows and 13 columns. A **1N4148** diode is soldered onto each switch to prevent false triggers when pressing multiple keys at once (ghosting).

There are two valid combinations of diode direction and scan polarity — the sections must match!

Variant A "Active-Low" (original, diode cathode toward row):

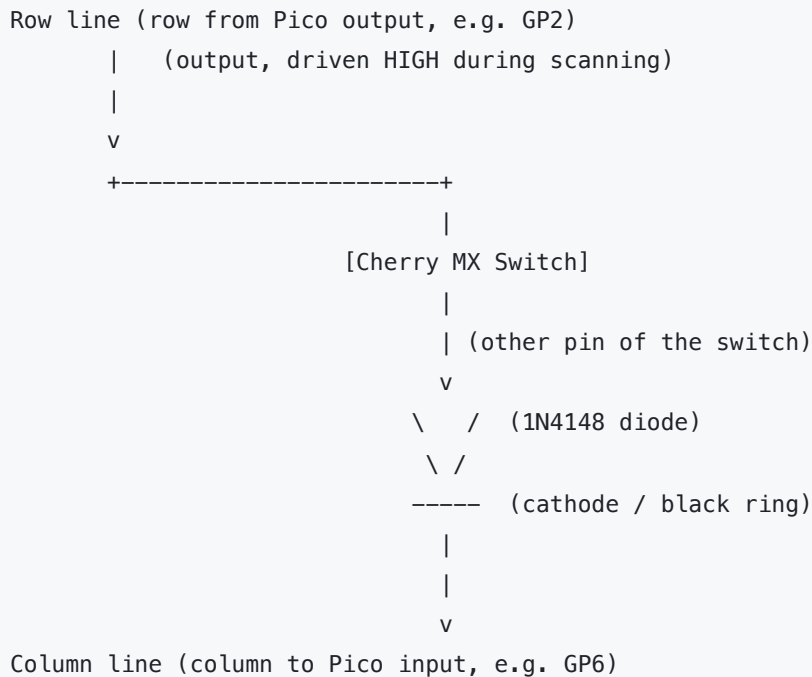
Diode: anode toward column, **cathode (black ring) toward row/switch**. Scanning: rows pulled LOW, columns with internal pull-ups, key detected at LOW.



Current path when pressing a key: **Column** (pull-up, HIGH) -> **Anode** -> **Cathode** -> **Switch** -> **Row** (LOW). The diode conducts current from the column to the low row and pulls the column LOW.

Variant B "Active-High" (standard on many PCBs, diode cathode toward column):

Diode: **cathode (black ring) toward column**, anode toward row/switch. Scanning: rows driven HIGH, columns with internal pull-downs, key detected at HIGH.



Current path when pressing a key: **Row** (HIGH) -> **Switch** -> **Anode** -> **Cathode** -> **Column** (pull-down, LOW).
The diode conducts current from the high row to the column and pulls the column HIGH.

Caution with existing wiring: If the diodes are already soldered and point toward the **Column** (original handbook), you do **not** need to re-solder them — the **C++ firmware (`src/main.cpp`)** already scans **Active-High** and works with that. The CircuitPython variant (`firmware/code.py`) scans Active-Low and requires diodes pointing toward the row.

Pin assignment on the Pico for the matrix:

- **Rows (outputs):** Row 0 (S row) = **GP4**, Row 1 (GO row) = **GP2**, Row 2 (Flash row) = **GP3**
- **Columns (inputs):** Column 1 to 13 on **GP6 to GP16, GP21, GP22**

4. Installation & flashing the bootloader

Variant A: CircuitPython (`firmware/code.py`)

1. Download the current stable `.uf2` file for the Raspberry Pi Pico from circuitpython.org/downloads.
2. Hold the **BOOTSEL** button on the Pico and plug it into the PC via USB.
3. Release the button. A drive named `RPI-RP2` opens.
4. Drag the downloaded `.uf2` file onto the drive. The Pico restarts and is now called **CIRCUITPY**.
5. Copy the source code from `firmware/code.py` directly onto the **CIRCUITPY** drive.

Variant B: PlatformIO / C++ (`platformio.ini` in the project root)

The C++ version is the **latency-optimized** variant (compiled to native ARM code) and uses the built-in USB-HID keyboard of the **earlephilhower Arduino core** — the same approach as the sister project `chamsys-encoder`.

```
# Open and build the project:
cd Documents/Arduino/chamsys-wing
pio run          # builds firmware.uf2

# Flash to the Pico (BOOTSEL hold-state required):
pio run -t upload
```

The generated file is at `.pio/build/pico/firmware.uf2` (alternatively by drag & drop onto the `RPI-RP2` drive after starting the Pico in BOOTSEL mode).

5. Setup in MagicQ

1. Start MagicQ, select **Setup** -> **View Settings**.
2. Switch to the **Keypad Encoders** tab.
3. Find the **MagicQ PC Keyboard Mode** line and change it to **Playback shortcuts**.
4. Ideally set your PC's operating system to the **US keyboard layout** for the DIY keyboard, so that special characters (`#`, ```, `~`, `\`, `|`, `'`) used by the wing are typed correctly by the Pico.
5. **Faders:** open the **Full Panel** window so that all 10 playback faders and the Grand Master are visible — the wing drags them there. If MagicQ shows a different window, open the Full Panel once before using the faders (the coordinates are calibrated for that layout).
6. **macOS tip:** If the Mac's physical Shift key occasionally drops out while the Pico is connected as an additional USB keyboard — that is known macOS behavior with multiple HID keyboards (the Pico injects Shift for characters like `#`). The firmware therefore sends lowercase letters (MagicQ evaluates key codes case-insensitively). If you don't want that at all, unplug the Pico when typing only on the Mac keyboard.
7. **Linux/Windows note:** The flash keys (F1–F10) work on the Mac, **not** on the PC (MagicQ-PC has no F-key binding in "Playback shortcuts"). The firmware therefore uses the **Test keys** (`\` `z` `x` `c` `v` `b` `n` `m` `,` `.`) instead, which work on every system and toggle (100% playback on/off). The firmware automatically compensates the toggling into **momentary flash**.

Project structure

```
chamsys-wing/
├─ README.md          # This handbook
├─ platformio.ini     # PlatformIO configuration (earlephilhower core)
├─ src/
│   └─ main.cpp       # C++ source (latency-optimized, USB-HID)
└─ firmware/
    └─ code.py        # CircuitPython source (alternative variant)
```